

Contents

1 - Mint (create the wallet) 2

2 - Spend (base M-of-N + delegated authorizations) 3

3 - Update withdrawals (per-UTxO) 4

4 - Publishing withdrawal scripts 5

5 - Close (admin burns the NFT) 6

1 - Mint (create the wallet)

The wallet is created by minting its identifying NFT — one token under the script’s own policy — into a fresh wallet UTxO at the wallet address. The admin must authorize the mint.

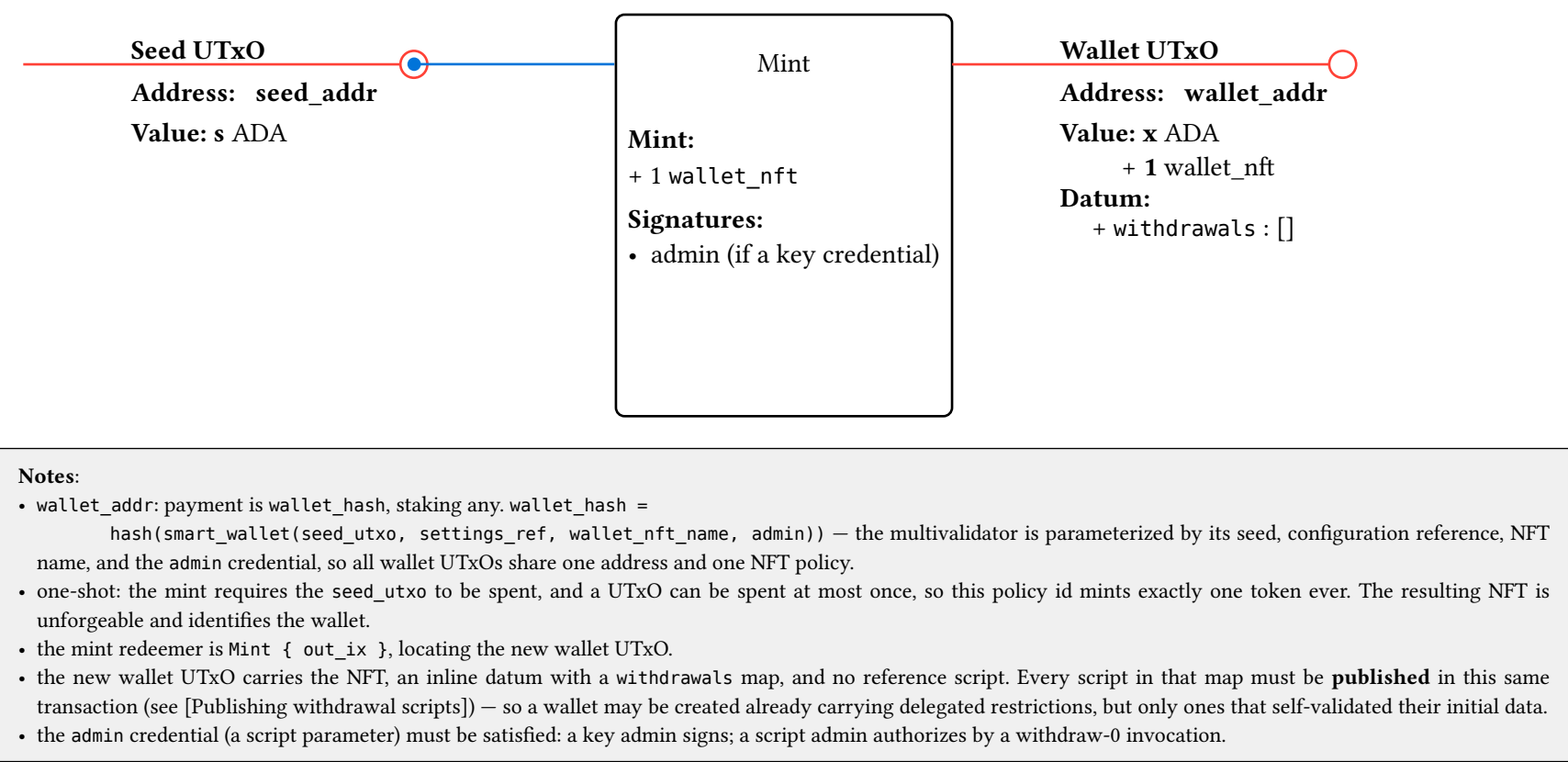
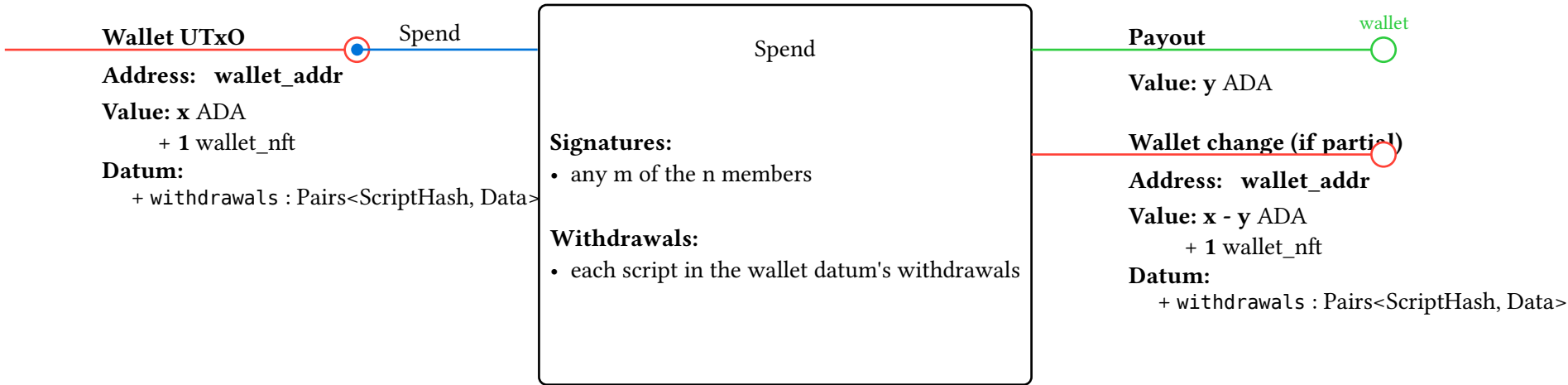


Figure 1: Mint (create the wallet)

2 - Spend (base M-of-N + delegated authorizations)

The wallet pays out. The spending script enforces its base M-of-N signature floor and forwards every other check to the withdrawal scripts named in the wallet UTxO's datum; the single spend redeemer asserts their conjunction.



- Notes:**
- `wallet_addr`: payment is `wallet_hash`, staking any. A wallet UTxO holds its identifying NFT plus ada and arbitrary native assets.
 - `wallet_hash = hash(smart_wallet(seed_utxo, settings_ref, wallet_nft_name, admin))`: the spending script is parameterized by its seed, configuration reference, NFT name, and the admin credential, so all wallet UTxOs share one address. Externalizing the members/threshold config lets it change without redeploying the script.
 - validity: the spend requires the wallet UTxO to carry its NFT (1 `wallet_nft` under `wallet_hash`) — only NFT-bearing UTxOs are valid smart wallets.
 - base M-of-N floor: the wallet's configuration supplies members and threshold, and the spend requires at least threshold of members present in `extra_signatories`. A member may itself be a script credential, so the floor can be another multisig, a DAO, and so on. This design is agnostic of how that configuration is sourced.
 - delegated authorizations: every `ScriptHash` key in the wallet UTxO's `withdrawals` datum field is a `withdraw-0` staking script that must run in this transaction; the `Data` value carries that script's own mutable state. The Spend redeemer asserts the conjunction — the M-of-N floor **and** all delegated checks — so no restriction can be silently omitted. The map is per-UTxO, so different wallet UTxOs may carry different restriction sets.
 - the delegated scripts are the extension point: spending limits, time locks, recipient allow-lists, oracle attestations, ... each is an independent withdrawal script observed here, none known to the spending script.
 - composability: the spend asserts only its own inputs and that the required withdrawals ran — never the total input or output counts, nor unrelated UTxOs. Any partial change keeps the NFT so the wallet stays valid.

Figure 2: Spend (base M-of-N + delegated authorizations)

3 - Update withdrawals (per-UTxO)

An authorized admin credential — a parameter of the spending script — rewrites a wallet UTxO’s delegated `withdrawals` list in place. Funds are untouched: same address, same value — only the restriction set changes.

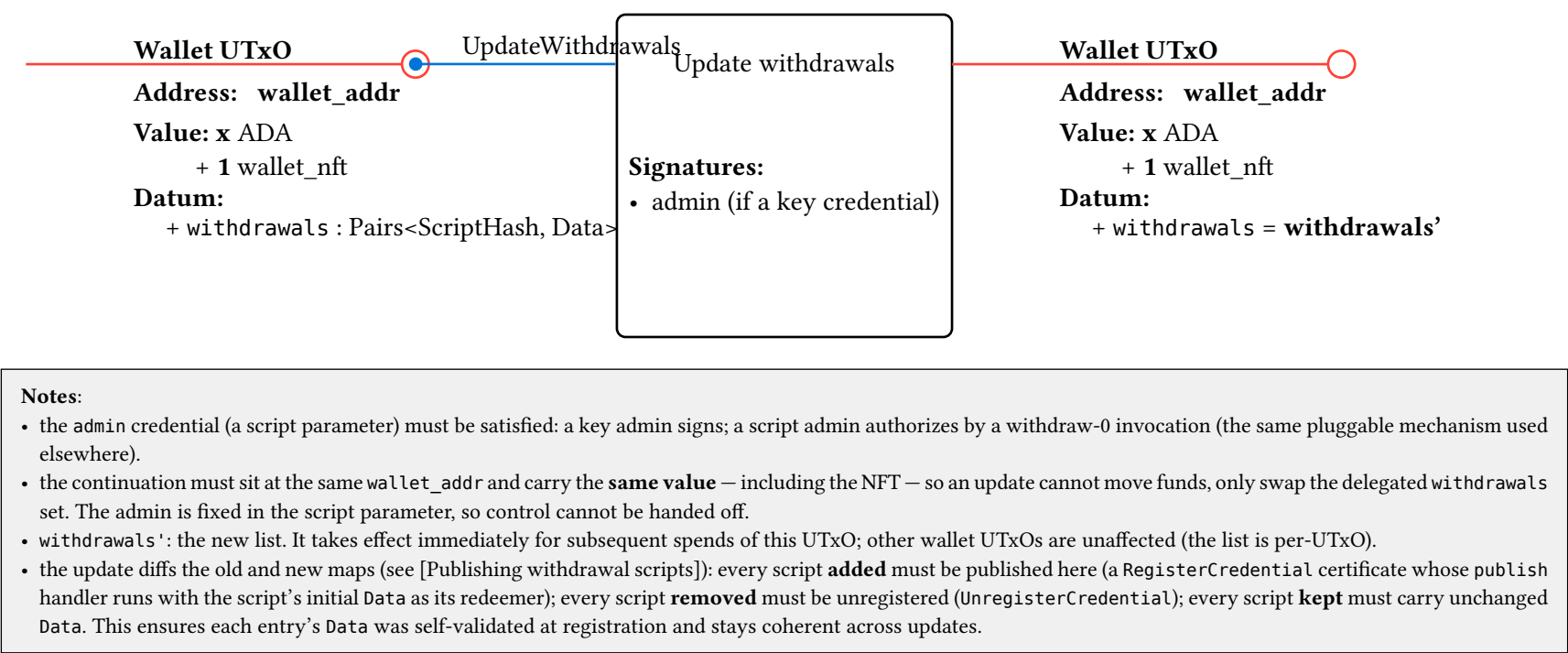
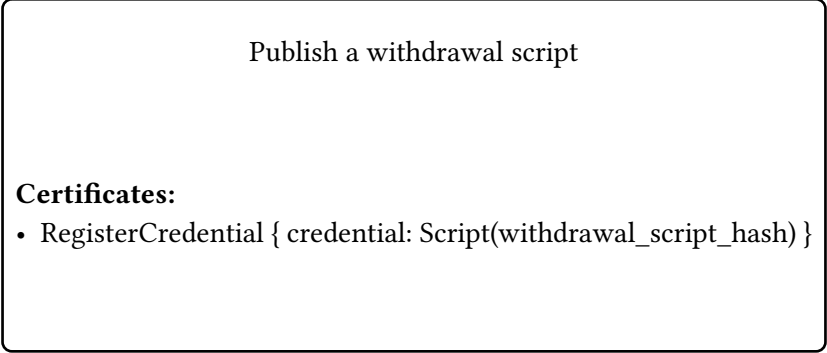


Figure 3: Update withdrawals (per-UTxO)

4 - Publishing withdrawal scripts

A *withdraw-0* script joins a wallet’s *withdrawals* map only after it is published: its stake credential is registered via a *RegisterCredential* certificate, which runs the script’s *publish* handler. The handler reads its own *initialData* from the wallet output’s datum and validates it, so no script is delegated without first self-validating its own starting state. Removing a script unregisters its credential, so re-adding it re-runs that validation.



- Notes:**
- the `RegisterCredential` certificate references the script’s stake credential, which triggers the script’s `publish` handler (CIP-69 certifying purpose).
 - the handler derives its own hash from the certificate and reads `withdrawals[own_hash]` out of the wallet output’s inline datum — the single source of truth for the script’s `Data` — and validates it.
 - the wallet only enforces that every script **added** to the `withdrawals` map has a matching `RegisterCredential` certificate (which implies its `publish` handler ran); all data validation is delegated to the script. `Mint` treats every script as added (there is no prior map).
 - removing a script requires an `UnregisterCredential` certificate for its credential, which also runs the script’s `publish` handler; the credential is freed so a later add can register it (and validate its data) again.
 - the reference `spending_limit` script is stateless (its bound is a compile-time parameter), so its `publish` handler only asserts the certificate registers or unregisters a script credential and that the script is listed in the wallet datum; a stateful script would additionally validate its `initialData` here.

Figure 4: Publishing a withdrawal script

5 - Close (admin burns the NFT)

The admin closes the wallet: the wallet UTxO is spent, releasing whatever funds remain, and the identifying NFT is burned. The Close spend redeemer requires the admin and the burn; the Burn mint redeemer permits it.

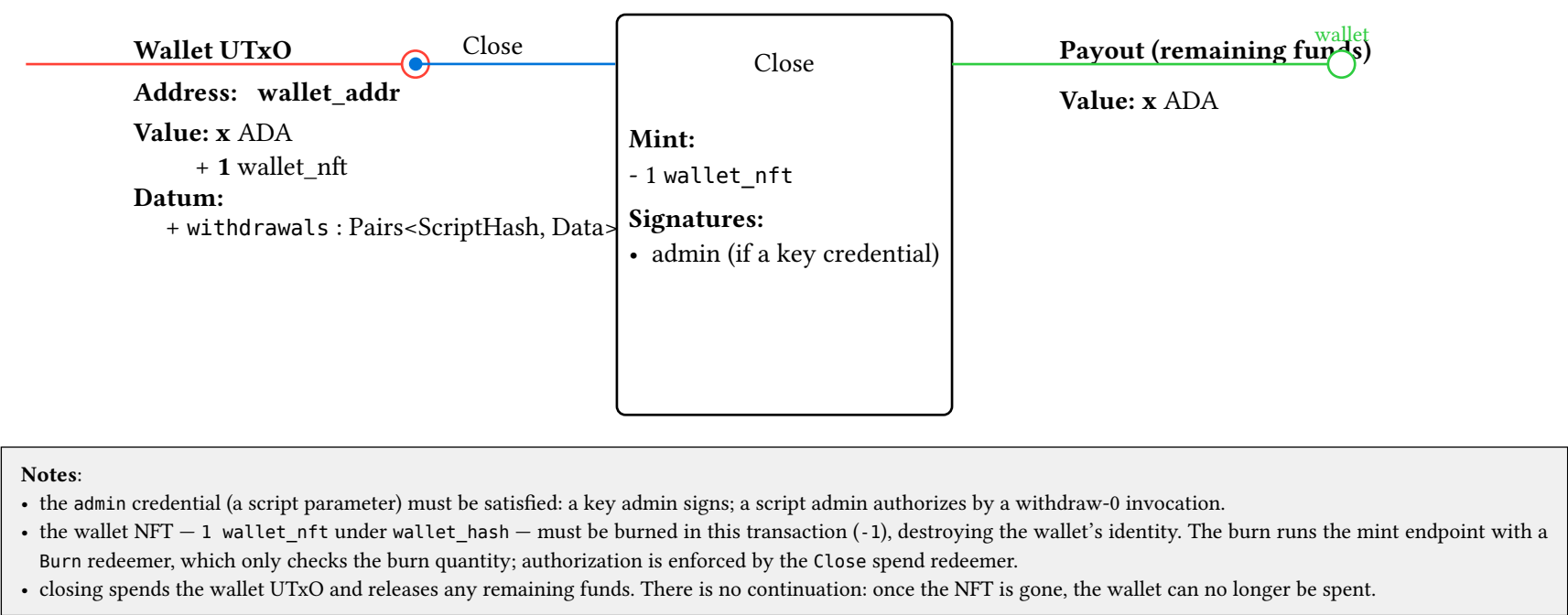


Figure 5: Close (admin burns the NFT)